Completado

PREPARACION · Paso 1 de 9

# Preparar ai-labs

Crea la carpeta del curso, valida las herramientas y abre el monorepo local.

## Antes de empezar

Esta preparación es común a los laboratorios. En Windows usamos exclusivamente **Git Bash dentro de Windows Terminal**, perfil **Workshop - Bash**. Los comandos `bash` de esta guía van en esa terminal. Los bloques `text` son prompts para **Copilot Chat en VS Code, modo Agent**. El portal permite leer, copiar y descargar, pero no ejecuta el producto ni certifica lo que declare el agente.

Completa estos cuatro pasos en orden con tu usuario habitual de Windows:

1. Crear tu carpeta del curso y la terminal Workshop - Bash.
2. Iniciar sesión en GitHub.
3. Validar las herramientas con un solo comando.
4. Crear tu repositorio desde el template y clonarlo.

Necesitas Windows 10 u 11, conexión a internet y Git for Windows, que incluye Git Bash. Para el primer paso abre **Git Bash** desde el menú Inicio; después usarás Workshop - Bash.

### 1. Crear tu carpeta del curso y la terminal

Ejecuta el instalador con la opción `curso` . Este comando pide tu matrícula y tu nombre, crea tu carpeta personal del taller y configura Windows Terminal para abrirla por defecto en el perfil Workshop - Bash. Al terminar, abre una pestaña en esa carpeta; allí clonarás tu repositorio. Conserva cualquier contenido existente.

Línea de comandos · bash Básico Avanzado

```
curl -fsSL https://bcp-ai-labs.pages.dev/install.sh | bash -s -- curso
```

```
Numero de matricula: 20261234
Nombre completo: Ana Pérez
```

La pestaña **Básico** pregunta los datos. La pestaña **Avanzado** evita las preguntas y permite elegir otra carpeta base.

El resultado esperado es:

text

```
C:/Users/Ana/Documents/20261234-ana-perez/
```

`curso` crea solo tu carpeta personal. `ai-labs/` se crea al clonar tu repositorio en el paso 4. Además:

- guarda matrícula y nombre en `.workshop-participant` y se detiene si la carpeta pertenece a otra matrícula;
- recuerda la ruta en `~/.workshop-course-dir` ;
- configura **Workshop - Bash**, crea sus accesos directos y conserva las preferencias existentes;
- abre una pestaña en la carpeta del curso;
- puede repetirse sin borrar trabajo; `--no-terminals` evita abrir las pestañas.

Si Windows Terminal ya estaba abierto, ciérralo y vuelve a abrirlo. En Workshop - Bash comprueba que estás en tu carpeta personal:

Línea de comandos · bash

```
printf 'Bash: %s\n' "$BASH_VERSION"
pwd -W
```

## 2. Iniciar sesión en GitHub

Usa la cuenta autorizada para el taller, no la del instructor. Elige GitHub.com, HTTPS y autenticación mediante navegador:

```
>_ Línea de comandos · bash
gh auth login
gh auth status
```

La identidad de los commits es independiente de esa sesión. Configura `git config user.name` y `git config user.email` con el instructor si están vacíos. Nunca pegues tokens en prompts. La preparación crea tu copia en GitHub y la clonación configura `origin` . Los ejercicios del Lab 01 usan ramas y commits locales; no requieren hacer push ni abrir un PR.

## 3. Validar las herramientas

Valida Git 2.40+, GitHub CLI con sesión, VS Code, JDK 21, Maven 3.9+ sobre Java 21, Python 3.13, Node.js 22.12+ y npm con el script del taller:

```
curl -fsSL https://bcp-ai-labs.pages.dev/install.sh | bash -s -- check --day 1
```

`check --day 1` no instala herramientas, no modifica tu carpeta y no comprueba Podman ni requisitos de otros días. Cada fila debe decir `OK` . Si falta algo, el instructor abre Git Bash como administrador y ejecuta:

```
curl -fsSL https://bcp-ai-labs.pages.dev/install.sh | SKIP_PODMAN=1 bash -s -- tools
```

`tools` instala las herramientas del equipo. El instalador y `check` no instalan ni comprueban extensiones de VS Code. `tools` no crea tu carpeta ni configura Workshop - Bash. Para eso usa `curso` con tu usuario habitual, como indica el paso 1. Si falla una herramienta, como Maven, la instalación queda incompleta. Después abre una terminal nueva y repite `check --day 1`. No ejecutes `all`, `day01`, `day02`, `labs`, `python-envs`, `mcp-images`, `inspector` ni `day02-mcp` para prepararlo.

## 4. Crear tu repositorio y clonarlo

Crear ai-labs en mi cuenta

El enlace abre GitHub con el template `NTTData-Academy/ai-labs`, el nombre `ai-labs`, su descripción y la visibilidad **Public** preseleccionados. En **Owner**, comprueba que figure tu cuenta personal, la misma que usaste en `gh auth login`. Conserva solo la rama principal y pulsa **Create repository**. Si ya tienes `ai-labs`, abre tu repositorio existente y comprueba que corresponde a este laboratorio.

El template es interno: necesitas acceso a NTTData-Academy para usarlo. Si GitHub no lo muestra, consulta al instructor.

En la pestaña Workshop - Bash que abrió `curso` ya estás en tu carpeta personal del taller. Clona allí tu repositorio con el siguiente comando, que en el portal incluye el usuario de tu sesión. Después entra en `ai-labs` y abre VS Code:

```
>_ Línea de comandos · bash
git clone https://github.com/user-bcp-student19/ai-labs.git
cd ai-labs
code .
```

Continúa solo si cada comando termina correctamente. La clonación crea `ai-labs/` dentro de tu carpeta personal. Si esa carpeta ya existe y contiene archivos, no los borres ni fuerces la clonación: comprueba con el instructor si debes reutilizar ese producto. No ejecutes `git init` ni clones `auth/`, `api/` o `web/` por separado. Un commit guarda cambios locales; clonar descarga el repositorio y configura su remoto.

## Cómo empezar

Tu copia del template contiene esta estructura. No hay repositorios Git anidados:

```text
ai-labs/                    # unica raiz Git y carpeta abierta en VS Code
├── auth/                      # Quarkus
├── api/                       # FastAPI
├── web/                       # Angular
├── README.md
├── AGENTS.md
├── RETO.md
└── docs/
    └── GOOD_PRACTICES.md
```

Estos son los documentos de entrada. `AGENTS.md` reúne las instrucciones de las tres plantillas en el formato compartido que reconocen los agentes compatibles. En VS Code con Copilot Agent, el archivo raíz se carga por defecto; no repitas su lectura en cada prompt. Revisa y acuerda sus reglas en Definir las reglas. `RETO.md` contiene candidatos que todavía debes contrastar con el código; no son hallazgos confirmados. `docs/GOOD_PRACTICES.md` ya viene con las plantillas originales: es una referencia de convenciones para acordar antes del refactor, no evidencia de trabajo realizado.

Durante el recorrido crearás `HALLAZGOS.md`, `PLAN.md`, `prompts/`, `docs/ENDPOINTS.md` y `docs/TESTING.md` cuando llegue su paso. No deben aparecer completos al inicio ni contener resultados inventados. Lee `README.md`, `AGENTS.md`, `RETO.md` y `docs/GOOD_PRACTICES.md`. Conserva las guías descargadas del portal fuera del producto y, al crear documentos generales, no sobrescribas las secciones aportadas por otros componentes.

Desde la raíz, con el trabajo anterior resguardado y el estado revisado, crea una única rama para el día:

```
>_ Línea de comandos · bash
git switch main
git switch -c day1/prompting
```

Si ya existe, usa `git switch day1/prompting` después de revisar su estado. No repitas la creación por componente. Abre cada servicio en una terminal aparte; Git se ejecuta siempre desde la raíz y los comandos de servicio y pruebas usan el directorio de su componente.

## Elige tu flujo

Elige la pestaña del componente para ver cómo iniciarlo.

**Autenticación · auth/** | **API de requerimientos · api/** | **Interfaz web · web/**

## API de requerimientos · Python y FastAPI

Revisa las rutas, el acceso a datos y la carga de archivos de `api/`.

En macOS o Linux, desde la raíz, crea el entorno solo si `api/.venv` no existe:

```
(cd api && python3.13 -m venv .venv)
```

Si ya existe, comprueba su versión y reutilízalo; no lo sobrescribas. En una terminal para este servicio, ejecuta desde la raíz:

```
(cd api && source .venv/bin/activate && python --version && python -m pip install -r requirements.txt -r requirements-dev.txt && python -m uvicorn app.main:app --host localhost --port 8000 --reload)
```

En Windows con Workshop - Bash, crea el entorno con `(cd api && py -3.13 -m venv .venv)` solo si falta y usa `source .venv/Scripts/activate` en lugar de `source .venv/bin/activate` .

Comprueba http://localhost:8000/docs. Guarda evidencias en los documentos generales de la raíz, con componente `api` e IDs `H-API-01` , `H-API-02` , etc. El código y las pruebas quedan en `api/` .

## Los 3 servicios

| COMPONENTE | PARA QUÉ SIRVE | DÓNDE COMPROBARLO |
|---|---|---|
| `auth/` , Quarkus | Gestiona el login y los tokens. | Salud del servicio, 8080 |
| `api/` , FastAPI | Expone los datos y las operaciones. | Documentación de la API, 8000 |
| `web/` , Angular | Muestra la aplicación y usa los otros dos servicios. | Abrir la aplicación, 4200 |

Para probar el login completo, inicia los tres servicios. Si trabajas solo uno, comprueba ese servicio y anota qué integraciones quedan sin probar. Mantén los servicios en localhost.

Conserva el bloque AMBITO de tu prompt. Permite leer los documentos generales y escribir evidencias en la raíz, pero solo leer y cambiar código del componente elegido. Las rutas de evidencia parten de `ai-labs` , por ejemplo `auth/ruta:línea` .

Si necesitas cambiar otro componente, registra la dependencia y coordina su alcance y pruebas con su responsable antes de ampliar el ámbito. Evita que dos agentes escriban el mismo documento general simultáneamente; integra sus aportes por turnos conservando los anteriores.

Leer, copiar o descargar el material

Completado

S1 · Paso 2 de 9

# Definir las reglas

Revisa AGENTS.md en la raíz: instrucciones persistentes y alcance del componente elegido.

Las instrucciones persistentes del producto viven en **`AGENTS.md`**, en la raíz de `ai-labs`. Es el formato compartido que reconocen los agentes compatibles; no es necesario repetir «lee las reglas» en cada prompt.

En VS Code con Copilot Agent, la lectura del `AGENTS.md` raíz está activada por defecto mediante `chat.useAgentsMdFile`. Si se desactiva o se usa otra herramienta, comprueba su mecanismo de instrucciones. Puedes consultar las referencias de la respuesta y las opciones de personalización de agentes de VS Code. El descubrimiento lo realiza la herramienta, no el modelo elegido.

## En tu proyecto

`ai-labs` ya incluye `AGENTS.md`, preparado a partir de las reglas de las tres plantillas originales. Revisa sus instrucciones comunes y la sección del componente elegido. Acuerda los ajustes antes de editar código; no reemplaces el archivo completo por el ejemplo de esta guía ni sobrescribas otras secciones.

- Código y pruebas solo en `auth/`, `api/` o `web/`, según el alcance elegido.
- Git desde la raíz; pruebas con el directorio del componente.
- Evidencias generales en la raíz, con componente e IDs globales, conservando aportes anteriores.
- Calidad y pruebas comprobadas con resultados reales. La aprobación humana precede a la integración y al checkpoint.

`docs/GOOD_PRACTICES.md` conserva las convenciones técnicas por stack. `AGENTS.md` no demuestra que una prueba se ejecutó ni sustituye la revisión humana. Los controles remotos no se ejecutan en el Día 1 local: conserva su configuración y registra esa limitación.

## Plantilla para adaptar a otro proyecto

text

```
# Instrucciones de proyecto


1. Analiza [el flujo / módulo principal] antes de cambiar [qué].
2. Conserva el contrato de [tu endpoint o interfaz pública].
3. Crea pruebas de caracterización antes de refactorizar [qué parte crítica].
4. No silencies [tus comprobaciones: linter, SAST, cobertura, etc.].
5. No agregues secretos reales; usa variables de entorno y datos de prueba aislados.
6. Separa [tus capas] en cambios revisables.
7. Meta de cobertura del código modificado: [tu %], con método y reporte.


## Clean code


8. Cada función/método tiene una responsabilidad.
```

```
9. Usa nombres que expliquen el propósito.
10. Revisa la duplicación y extrae lógica compartida cuando corresponda.
11. Evita anidamiento profundo; considera retornos anticipados.
12. Usa constantes con nombre para valores de dominio relevantes.
```

## Ejemplo para la sección auth

Este ejemplo ilustra la adaptación; no reemplaza las instrucciones existentes ni acredita resultados.

text

```
## auth


1. Analiza el flujo completo antes de cambiar autenticación.
2. Conserva el contrato de POST /auth/login.
3. Crea pruebas de caracterización antes de refactorizar persistencia o token.
4. No silencies CodeQL, Dependency Review ni JaCoCo.
5. No agregues secretos reales; usa variables de entorno y datos de prueba aislados.
6. Separa API, servicio, repositorio y seguridad en cambios revisables.
7. Meta de cobertura del código modificado: 85% o superior.
   Distingue cobertura global y del cambio; declara el método y sus límites.
```

Referencias: AGENTS.md y personalización de instrucciones en VS Code.

Leer, copiar o descargar el material

Completado

S1 · Paso 3 de 9

# Construir y comprobar un prompt

Compara pedidos, adapta criterios y verifica la respuesta.

# S1: construir un pedido y comprobar la respuesta

Un prompt reduce ambigüedades y hace explícitas tus decisiones. No garantiza una respuesta correcta. El agente puede leer mal el código, omitir un caso o afirmar que ejecutó una prueba que nunca corrió. Tu trabajo incluye comprobarlo.

## Construir un pedido

Usaremos seis partes. Ámbito indica dónde puede actuar; las otras cinco explican qué necesita saber y qué resultado esperas.

| PARTE | QUÉ APORTA | EJEMPLO DE AI-LABS |
|---|---|---|
| Ámbito | Delimita lectura, escritura y rutas relativas | Código solo en `auth/`, evidencias generales en raíz, sin tocar código de API ni frontend |
| Contexto | Aporta evidencia y decisiones existentes | Lee el contrato de `POST /auth/login`, `RETO.md` y reglas |
| Objetivo | Define el resultado buscado | Encontrar cómo se construye la consulta de login |
| Restricciones | Expresa límites al trabajo | No editar código ni cambiar el contrato |
| Criterios de aceptación | Permite contrastar el resultado | Citar archivo y línea, distinguir hipótesis y hallazgo confirmado |
| Formato de respuesta | Facilita consumir y revisar el resultado | Tabla con riesgo, evidencia y comprobación pendiente |

 Las seis partes se reúnen en un pedido que se puede revisar

El diagrama reúne las decisiones antes de enviar el pedido. Contexto no significa pegar todo el repositorio: selecciona archivos que permitan resolver la tarea dentro del ámbito.

**Comprobación:** en el prompt de abajo señala qué frase limita archivos, cuál aporta contexto y cuál permite refutar la respuesta. Si una frase no tiene una función clara, reescríbela.

## Comparar dos pedidos sobre la misma tarea

Pedido ambiguo, para analizar en clase antes de ejecutarlo:

```text
Revisa si el login tiene SQL injection.
```

No dice qué login, qué puede modificar ni qué evidencia necesitas. Puede producir una respuesta genérica o una corrección que no pediste. Mejora el mismo pedido:

```text
AMBITO
Workspace ai-labs abierto en su raiz. Lee codigo solo en auth/.
Todas las rutas parten de la raiz. Lee README.md, RETO.md y
docs/ generales. Este pedido es de solo lectura; no escribas archivos.
Git se ejecuta desde la raiz; las pruebas propuestas usan cwd auth/.
No leas ni modifiques codigo de api/ ni web/. Usa IDs H-AUTH-NN y componente auth.


CONTEXTO
```

```
ai-labs autentica mediante POST /auth/login. Lee README.md,
RETO.md y docs/ENDPOINTS.md si existe. Localiza la ruta desde el recurso
HTTP hasta la consulta a SQLite. No des por cierta la lista de RETO.md.

OBJETIVO
Determinar si entradas del login llegan concatenadas a una consulta SQL.

RESTRICCIONES
No edites codigo ni ejecutes ataques contra servicios remotos. Usa solo
codigo y datos ficticios locales. Si falta un archivo o no puedes ejecutar
```

El objetivo sigue siendo revisar SQL injection en login. Cambió la precisión: el ámbito evita mezclar implementaciones, el contexto dirige la lectura, los límites evitan ediciones prematuras y los criterios exigen una cadena de evidencia. El formato deja visibles las incertidumbres. Pedir evidencia mejora la revisión; no demuestra que las citas sean verdaderas.

 Un pedido ambiguo recibe contexto, límites y criterios hasta volverse comprobable

Cada paso elimina una ambigüedad distinta. Una respuesta más larga no prueba que el pedido mejoró: debe permitir decidir si la afirmación es correcta.

**Actividad:** agrega al pedido ambiguo una sola parte cada vez. Antes de enviar, escribe qué suposición deja de ser necesaria. Compara con un compañero qué información sigue faltando. No envíes el pedido ambiguo si el modo Agent tiene permisos de edición amplios.

## Comprobar contra código y pruebas

1. Abre las rutas citadas dentro de `auth/`. Comprueba las líneas y sigue el dato desde el request hasta la consulta. Una línea con SQL no demuestra por sí sola que sea alcanzable con entrada de usuario.

2. Lee el contrato observado y separa comportamiento existente de comportamiento deseado. Una prueba de caracterización registra el primero; una prueba de seguridad debe demostrar la corrección, aunque falle antes del cambio.
3. Ejecuta los comandos de pruebas y cobertura en tu terminal. Compara salida, código de salida y reportes con lo afirmado por el agente.
4. Revisa `git diff main -- auth/` desde la raíz de `ai-labs`, y revisa las secciones de auth en los documentos generales. Verifica archivos nuevos con `git status --short`. Confirma que el cambio corrige la causa y que la prueba fallaría ante el defecto original.
5. Si una afirmación carece de evidencia, mantenla como hipótesis. Registra el límite y formula el siguiente pedido antes de marcar un hallazgo como resuelto.

 La respuesta se contrasta con código, diff y pruebas; la evidencia faltante genera el siguiente pedido

La revisión devuelve información al prompt. Si un comando falla por el entorno, eso es un resultado de ejecución; no prueba que el código sea correcto ni que el hallazgo esté corregido.

**Comprobación:** toma una fila de la respuesta. ¿Otra persona puede repetir el comando y localizar las líneas sin preguntarte nada? Señala qué evidencia le falta.

## Seguimiento cuando falta evidencia

text

```
Conserva el AMBITO anterior: codigo solo en auth/, documentos generales
de la raiz en lectura, sin editar archivos.
Afirmaste que el login usa consultas parametrizadas, pero no citaste el
punto donde se enlazan los parametros y no ejecutaste pruebas.
No edites codigo. Muestra archivo y linea de la construccion y ejecucion
```

```
de la consulta y sigue el origen de sus valores. Indica si esa evidencia
confirma o contradice tu afirmacion anterior.
Propone el test que distinguiria concatenacion de parametros enlazados.
Separa codigo leido, pruebas ejecutadas y pruebas propuestas. Si falta
acceso, nombra el bloqueo. No marques H-AUTH-NN como Resuelto por esta respuesta.
```

Este seguimiento nombra la afirmación cuestionada y pide la evidencia que la resolvería. No basta con decir "sé más preciso".

## Tu adaptación y su justificación

Elige un componente y una pregunta confirmable del ejercicio. Usa su plantilla, o las de API y frontend.

- Decide el foco. Por ejemplo, en API: "¿una entrada de nombre de archivo puede salir del directorio de uploads?"; en frontend: "¿se muestra un error cuando el servicio de autenticación no responde?".
- Completa las seis partes sin inventar rutas de código. Usa el discovery para localizar las que aún no conoces.
- Define dos criterios verificables y justifica por qué detectan el riesgo. "Código limpio" no basta. "Ante el servicio de autenticación no disponible se muestra un mensaje y no se conserva sesión activa, según contrato acordado" sí puede contrastarse con una prueba.
- Pide revisión de tu prompt a un compañero. Ejecútalo después de ajustar ámbito y permisos. Contrasta una afirmación con código y una con prueba; registra lo que no pudiste comprobar.
- Guarda en `prompts/` general en la raíz, con el componente en el nombre el pedido inicial, la versión adaptada, tu justificación y el seguimiento. Explica qué cambiaste después de recibir la respuesta y qué duda sigue abierta.

La plantilla contiene decisiones del estudiante, no datos que debas adivinar. Un ID como `H-AUTH-02` se usa solo si existe. Un umbral debe venir de `AGENTS.md` y medirse; un formato de tabla debe ayudar a tu revisión.

## Fuentes de los diagramas

Los SVG locales se generaron con Pele 1.6.0. No necesitas instalarlo para leerlos. Puedes descargar las fuentes editables construir.mmd, mejorar.mmd y revisar.mmd, o ver los SVG.

Para regenerarlos, el mantenedor instala Pele fuera del producto, por ejemplo en `output/day1/pele`, con `npm install --prefix output/day1/pele --no-package-lock pele@1.6.0`, y ejecuta `node tools/day1-diagrams.mjs`. Esto no forma parte de la preparación del estudiante.

Abre ai-labs en VS Code desde la raíz de ai-labs/. Conserva AMBITO: evidencias generales en la raíz y código solo en auth/, api/ o web/, según el componente elegido. Tu elección se mantiene en los demás pasos de este día.

# Adaptar un prompt de ai-labs · API de requerimientos

Lee construir y comprobar un pedido antes de copiar. Conserva AMBITO. Completa las decisiones entre corchetes y explica por qué tus criterios detectarían el problema. La estructura no garantiza exactitud.

Autenticación · auth/ **API de requerimientos · api/** Interfaz web · web/

```
AMBITO
Workspace: ai-labs, abierto en su raiz. Todas las rutas son relativas
a esa raiz. Lee el codigo solo de api/; escribe codigo, configuracion y
pruebas solo en api/. No leas ni modifiques codigo de otros componentes.
Lee README.md, RETO.md y docs/GOOD_PRACTICES.md. Lee
HALLAZGOS.md, PLAN.md y los demas documentos de docs/ solo si existen.
Escribe evidencias solo en HALLAZGOS.md, PLAN.md, prompts/,
docs/ENDPOINTS.md, docs/TESTING.md y docs/GOOD_PRACTICES.md de la raiz.
Cuando este paso pida un artefacto ausente, crealo. Si ya existe, conserva
```

```
las filas y secciones de otros componentes. No las sobrescribas.
Cada hallazgo lleva componente api e ID H-API-NN, por ejemplo H-API-01.
Cita codigo como api/ruta:linea. No renumeres IDs existentes.
Ejecuta Git desde ai-labs/; ejecuta pruebas con cwd api/.
Coordina dependencias externas sin ampliar AMBITO. No inventes aprobaciones
humanas ni resultados de pruebas. Si falta evidencia, registra el pendiente.

CONTEXTO
Lee README.md y RETO.md. Usa HALLAZGOS.md y docs/ENDPOINTS.md
```

Ejemplos para completar las decisiones de este componente:

- Pregunta: "¿qué ocurre en carga de archivos ante una entrada inválida?". Concreta el dato inválido según su contrato.
- Resultado: "localizar el flujo y proponer una comprobación local sin editar".
- Criterios: "cada afirmación cita archivo y línea" y "se identifica una prueba de entrada válida y otra inválida con resultado esperado según el contrato".
- Formato: "tabla de afirmación, evidencia, prueba propuesta y límite". Permite revisar una fila a la vez.

Cuando pases a corrección, selecciona IDs reales de HALLAZGOS.md, por ejemplo `H-API-02, H-API-04` solo si existen. Justifica su riesgo y el motivo de dejar otros pendientes. Si `prompts/` no existe, créalo. Guarda el prompt adaptado y esa justificación en `prompts/`, con el componente en el nombre del archivo. Coordina un cambio de contrato con los responsables de los consumidores antes de ampliar el ámbito.

✓ Completado

S2 · Paso 4 de 9

# 1. Encontrar hallazgos

Lee RETO.md y contrasta cada hallazgo contra el código sin modificarlo.

Abre ai-labs en VS Code desde la raíz de ai-labs/. Conserva AMBITO: evidencias generales en la raíz y código solo en auth/, api/ o web/, según el componente elegido. Tu elección se mantiene en los demás pasos de este día.

# Prompt 01 · Discovery de ai-labs · API de requerimientos

Autenticación · auth/ | **API de requerimientos · api/** | Interfaz web · web/

```
AMBITO
Workspace: ai-labs, abierto en su raiz. Todas las rutas son relativas
a esa raiz. Lee el codigo solo de api/; escribe codigo, configuracion y
pruebas solo en api/. No leas ni modifiques codigo de otros componentes.
Lee README.md, RETO.md y docs/GOOD_PRACTICES.md. Lee
HALLAZGOS.md, PLAN.md y los demas documentos de docs/ solo si existen.
Escribe evidencias solo en HALLAZGOS.md, PLAN.md, prompts/,
docs/ENDPOINTS.md, docs/TESTING.md y docs/GOOD_PRACTICES.md de la raiz.
Cuando este paso pida un artefacto ausente, crealo. Si ya existe, conserva
las filas y secciones de otros componentes. No las sobrescribas.
Cada hallazgo lleva componente api e ID H-API-NN, por ejemplo H-API-01.
Cita codigo como api/ruta:linea. No renumeres IDs existentes.
Ejecuta Git desde ai-labs/; ejecuta pruebas con cwd api/.
Coordina dependencias externas sin ampliar AMBITO. No inventes aprobaciones
humanas ni resultados de pruebas. Si falta evidencia, registra el pendiente.
```

```
CONTEXTO

Trabaja solo las filas y secciones de api en los documentos generales.
```

Completado

S3 · Paso 5 de 9

# 2. Planificar y refactorizar

Genera PLAN.md antes de corregir y actualiza HALLAZGOS.md con evidencia.

Abre ai-labs en VS Code desde la raíz de ai-labs/. Conserva AMBITO: evidencias generales en la raíz y código solo en auth/, api/ o web/, según el componente elegido. Tu elección se mantiene en los demás pasos de este día.

# Prompt 02 · Refactor de ai-labs · API de requerimientos

Resuelve **cualquier tipo** de hallazgo (Vulnerabilidad, Bug, Refactor o Sugerencia) — el nombre "refactor" es por la actividad de esta sesión (corregir código), no filtra por la columna "Tipo" de `HALLAZGOS.md` .

Este prompt es para el **primer lote**: los hallazgos que salieron de `1-discovery.md` , apenas terminas de encontrarlos. Para cualquier hallazgo que aparezca después (uno que quedó pendiente, uno nuevo de `3-review.md` , uno que encontraste tú), usa `resolver-item.md` en vez de repetir este — mismo mecanismo, sin volver a explicar todo el contexto.

Ejecuta esto para resolver los hallazgos marcados "Pendiente" en `HALLAZGOS.md` . Tú eliges el alcance: puedes priorizar los **2 o 3 de mayor severidad** si el tiempo apremia (no hace falta resolver los 7 de `RETO.md` en la sesión — priorizar también es parte de lo que se practica), o resolverlos **todos** si te alcanza. Va en dos pasos: primero el plan, después la implementación. No saltes el paso 1.

# Paso 1 — Generar el plan (no tocar código)

Autenticación · auth/ | API de requerimientos · api/ | Interfaz web · web/

```
AMBITO
Workspace: ai-labs, abierto en su raiz. Todas las rutas son relativas
a esa raiz. Lee el codigo solo de api/; escribe codigo, configuracion y
pruebas solo en api/. No leas ni modifiques codigo de otros componentes.
Lee README.md, RETO.md y docs/GOOD_PRACTICES.md. Lee
HALLAZGOS.md, PLAN.md y los demas documentos de docs/ solo si existen.
Escribe evidencias solo en HALLAZGOS.md, PLAN.md, prompts/,
docs/ENDPOINTS.md, docs/TESTING.md y docs/GOOD_PRACTICES.md de la raiz.
Cuando este paso pida un artefacto ausente, crealo. Si ya existe, conserva
las filas y secciones de otros componentes. No las sobrescribas.
Cada hallazgo lleva componente api e ID H-API-NN, por ejemplo H-API-01.
Cita codigo como api/ruta:linea. No renumeres IDs existentes.
Ejecuta Git desde ai-labs/; ejecuta pruebas con cwd api/.
Coordina dependencias externas sin ampliar AMBITO. No inventes aprobaciones
humanas ni resultados de pruebas. Si falta evidencia, registra el pendiente.

CONTEXTO
Trabaja solo las filas y secciones de api en los documentos generales.
```

Antes de pegar el siguiente bloque, una persona revisa PLAN.md y registra aprobador, fecha, H-API-NN seleccionados, contratos y pruebas en ese archivo. La IA no puede darse esa aprobacion. Si afecta a otro componente, registra la

dependencia, coordina su responsable y solicita un ambito explicito aprobado. Conserva los limites del AMBITO actual hasta entonces.

## Paso 2 — Implementar el plan aprobado

Autenticación · auth/ API de requerimientos · api/ Interfaz web · web/

```
AMBITO
Workspace: ai-labs, abierto en su raiz. Todas las rutas son relativas
a esa raiz. Lee el codigo solo de api/; escribe codigo, configuracion y
pruebas solo en api/. No leas ni modifiques codigo de otros componentes.
Lee README.md, RETO.md, HALLAZGOS.md, PLAN.md y
docs/GOOD_PRACTICES.md. Consulta los demas documentos de docs/ si existen.
Escribe evidencias solo en HALLAZGOS.md, PLAN.md, prompts/,
docs/ENDPOINTS.md, docs/TESTING.md y docs/GOOD_PRACTICES.md de la raiz,
conservando las filas y secciones de otros componentes. No los sobrescribas.
Cada hallazgo lleva componente api e ID H-API-NN, por ejemplo H-API-01.
Cita codigo como api/ruta:linea. No renumeres IDs existentes.
Ejecuta Git desde ai-labs/; ejecuta pruebas con cwd api/.
Coordina dependencias externas sin ampliar AMBITO. No inventes aprobaciones
humanas ni resultados de pruebas. Si falta evidencia, registra el pendiente.

CONTEXTO
Trabaja solo las filas y secciones de api en los documentos generales.
Conserva contenido previo de otros componentes y el historial de este componente.
```

Si `HALLAZGOS.md` tiene más hallazgos "Pendiente" que quieras resolver después de estos, o algo que surgió recién (un bug, una sugerencia de review), usa resolver-item.md — mismo mecanismo, para un item a la vez o varios, sin necesidad de repetir todo este prompt.

Completado

S2-REVIEW · Paso 6 de 9

# 3. Revisar las correcciones

Otra persona confirma o reabre hallazgos con un motivo comprobable.

Abre ai-labs en VS Code desde la raíz de ai-labs/. Conserva AMBITO: evidencias generales en la raíz y código solo en auth/, api/ o web/, según el componente elegido. Tu elección se mantiene en los demás pasos de este día.

# Prompt 03 · Revisión de ai-labs · API de requerimientos

Autenticación · auth/ | **API de requerimientos · api/** | Interfaz web · web/

```
AMBITO
Workspace: ai-labs, abierto en su raiz. Todas las rutas son relativas
a esa raiz. Lee el codigo solo de api/; escribe codigo, configuracion y
pruebas solo en api/. No leas ni modifiques codigo de otros componentes.
Lee README.md, RETO.md y docs/GOOD_PRACTICES.md. Lee
HALLAZGOS.md, PLAN.md y los demas documentos de docs/ solo si existen.
Escribe evidencias solo en HALLAZGOS.md, PLAN.md, prompts/,
docs/ENDPOINTS.md, docs/TESTING.md y docs/GOOD_PRACTICES.md de la raiz.
```

```
Cuando este paso pida un artefacto ausente, crealo. Si ya existe, conserva
las filas y secciones de otros componentes. No las sobrescribas.
Cada hallazgo lleva componente api e ID H-API-NN, por ejemplo H-API-01.
Cita codigo como api/ruta:linea. No renumeres IDs existentes.
Ejecuta Git desde ai-labs/; ejecuta pruebas con cwd api/.
Coordina dependencias externas sin ampliar AMBITO. No inventes aprobaciones
humanas ni resultados de pruebas. Si falta evidencia, registra el pendiente.

CONTEXTO
La evidencia es git diff main -- api/ desde la raiz ai-labs/,
```

Si encontraste algo que no estaba en `HALLAZGOS.md` (un bug nuevo, una sugerencia importante), agrégalo ahí mismo como una fila nueva con estado "Pendiente" — no lo dejes solo mencionado en el chat. Se resuelve después con resolver-item.md, igual que cualquier otro item.

Completado

S4 · Paso 7 de 9

# 4. Documentar el contrato final

Actualiza docs/ENDPOINTS.md con el comportamiento comprobado.

Abre ai-labs en VS Code desde la raíz de ai-labs/. Conserva AMBITO: evidencias generales en la raíz y código solo en auth/, api/ o web/, según el componente elegido. Tu elección se mantiene en los demás pasos de este día.

# Prompt 04 · Documentar el contrato de ai-labs · API de requerimientos

Ejecuta esto después de revisar el lote implementado, aunque queden hallazgos "Pendiente". Documenta comportamiento real, límites y riesgos abiertos.

Autenticación · auth/ | API de requerimientos · api/ | Interfaz web · web/

```
AMBITO
Workspace: ai-labs, abierto en su raiz. Todas las rutas son relativas
a esa raiz. Lee el codigo solo de api/; escribe codigo, configuracion y
pruebas solo en api/. No leas ni modifiques codigo de otros componentes.
Lee README.md, RETO.md y docs/GOOD_PRACTICES.md. Lee
HALLAZGOS.md, PLAN.md y los demas documentos de docs/ solo si existen.
Escribe evidencias solo en HALLAZGOS.md, PLAN.md, prompts/,
docs/ENDPOINTS.md, docs/TESTING.md y docs/GOOD_PRACTICES.md de la raiz.
Cuando este paso pida un artefacto ausente, crealo. Si ya existe, conserva
las filas y secciones de otros componentes. No las sobrescribas.
Cada hallazgo lleva componente api e ID H-API-NN, por ejemplo H-API-01.
Cita codigo como api/ruta:linea. No renumeres IDs existentes.
Ejecuta Git desde ai-labs/; ejecuta pruebas con cwd api/.
Coordina dependencias externas sin ampliar AMBITO. No inventes aprobaciones
humanas ni resultados de pruebas. Si falta evidencia, registra el pendiente.

CONTEXTO
Trabaja solo las filas y secciones de api en los documentos generales.
```

Completado

S4 · Paso 8 de 9

# 5. Completar las pruebas

Ejecuta las pruebas, cubre los riesgos y documenta resultados reales.

Abre ai-labs en VS Code desde la raíz de ai-labs/. Conserva AMBITO: evidencias generales en la raíz y código solo en auth/, api/ o web/, según el componente elegido. Tu elección se mantiene en los demás pasos de este día.

# Prompt 05 · Completar las pruebas de ai-labs · API de requerimientos

Continúa después del paso 4, Documentar el contrato final. Las pruebas básicas ya se ejecutaron durante la implementación; aquí completas los casos que faltan y compruebas sus resultados.

## 1. Completar las pruebas con Copilot

Copia el siguiente prompt en Copilot, en modo Agent, con `ai-labs/` abierto desde su raíz. Revisa los cambios propuestos y conserva las secciones de otros componentes en los documentos generales.

Autenticación · auth/ **API de requerimientos · api/** Interfaz web · web/

```
AMBITO
Workspace: ai-labs, abierto en su raiz. Todas las rutas son relativas
```

```
a esa raiz. Lee el codigo solo de api/; escribe codigo, configuracion y
pruebas solo en api/. No leas ni modifiques codigo de otros componentes.
Lee README.md, RETO.md y docs/GOOD_PRACTICES.md. Lee
HALLAZGOS.md, PLAN.md y los demas documentos de docs/ solo si existen.
Escribe evidencias solo en HALLAZGOS.md, PLAN.md, prompts/,
docs/ENDPOINTS.md, docs/TESTING.md y docs/GOOD_PRACTICES.md de la raiz.
Cuando este paso pida un artefacto ausente, crealo. Si ya existe, conserva
las filas y secciones de otros componentes. No las sobrescribas.
Cada hallazgo lleva componente api e ID H-API-NN, por ejemplo H-API-01.
Cita codigo como api/ruta:linea. No renumeres IDs existentes.
Ejecuta Git desde ai-labs/; ejecuta pruebas con cwd api/.
Coordina dependencias externas sin ampliar AMBITO. No inventes aprobaciones
humanas ni resultados de pruebas. Si falta evidencia, registra el pendiente.

CONTEXTO
Trabaja solo las filas y secciones de api en los documentos generales.
```

## 2. Ejecutar las pruebas en tu terminal

Cuando Copilot termine, revisa los tests y compáralos con el contrato del paso 4. Deben comprobar los casos válidos, los límites y los errores documentados. Una respuesta del agente no sustituye la ejecución.

Abre la terminal integrada de VS Code con el perfil Workshop - Bash, desde la raíz de `ai-labs/` . Si el servicio sigue activo en otra terminal y ocupa los puertos o la base de pruebas, detenlo con Ctrl+C en esa terminal.

Comprueba primero que `api/requirements-dev.txt` incluye pytest y pytest-cov, y ya preparaste el entorno `.venv` de este componente. Si falta un script, una dependencia o la configuración de cobertura, pide a Copilot que lo revise dentro de

`api/` y registra el bloqueo hasta resolverlo.

Ejecuta este bloque. Los paréntesis entran en `api/` solo durante el comando; al terminar, tu terminal sigue en la raíz de `ai-labs/` .

En Workshop - Bash de Windows:

```
(cd api && source .venv/Scripts/activate && python -m pytest --cov=app --cov-report=term-missing --cov-report=html --cov-fail-under=85)
```

En macOS o Linux, usa este bloque en su lugar:

```
(cd api && source .venv/bin/activate && python -m pytest --cov=app --cov-report=term-missing --cov-report=html --cov-fail-under=85)
```

Al terminar el bloque, consulta de inmediato el código de salida:

```
echo $?
```

Debe ser `0` . Si es distinto de `0` , conserva el error, revisa su causa y repite las pruebas afectadas después de corregirla. No cambies una expectativa válida para ocultar un bug ni rebajes un gate fallido.

## 3. Comprobar cobertura y guardar evidencia

Lee el resumen de cobertura de la terminal y abre `api/htmlcov/index.html` . El gate `--cov-fail-under=85` mide toda `app` , no solo el código modificado. Si el reporte no se generó, registra que falta; no declares una cobertura sin medirla.

La meta es al menos 85 % del código modificado. Registra por separado la cobertura global y la del cambio, con líneas cubiertas, líneas medidas y método de cálculo. El porcentaje global por sí solo no demuestra la cobertura del cambio. Si no puedes medir esta última, declara la limitación.

Crea o actualiza la sección `api` de `docs/TESTING.md` en la raíz, conservando las demás secciones. Por cada ejecución, guarda:

- Componente e IDs de los hallazgos comprobados.
- Endpoint o flujo, caso, tipo de prueba y resultado esperado.
- Comando, directorio de ejecución, fecha y código de salida real.
- Resumen de resultados y ruta del reporte de cobertura.
- Fallos y pendientes, separados de las pruebas ejecutadas y de las propuestas.

No registres checks remotos de GitHub como ejecutados: este recorrido es local. Si agregaste pruebas o código, vuelve a revisar el diff con la persona revisora antes del cierre. Continúa a Cerrar y entregar al Día 2 cuando tengas la evidencia; Resolver un hallazgo adicional es opcional si todavía queda uno por atender.

Completado

CIERRE · Paso opcional

Opcional

# Resolver un hallazgo adicional

Usa este prompt después del paso 2 solo si queda un hallazgo del componente por resolver. Este paso es opcional.

Abre ai-labs en VS Code desde la raíz de ai-labs/. Conserva AMBITO: evidencias generales en la raíz y código solo en auth/, api/ o web/, según el componente elegido. Tu elección se mantiene en los demás pasos de este día.

# Resolver un hallazgo adicional de ai-labs · API de requerimientos

Este prompt no es específico de seguridad ni de refactor: sirve para **cualquier item "Pendiente" de** `HALLAZGOS.md` , sin importar si llegó ahí por el discovery inicial ( `1-discovery.md` ), por un code review ( `3-review.md` ), o porque lo agregaste a mano al encontrar un bug o una sugerencia. Es lo mismo que hace `2-refactor.md` , pero para todo lo que aparece **después** de ese primer lote — no repitas `2-refactor.md` , usa este.

Puedes indicar **uno o varios items a la vez** (por ejemplo: "resuelve H-API-02 y H-API-04, solo si existen", "resuelve los 3 de mayor severidad", o "sigue con el resto de HALLAZGOS.md"). Si no indicas ninguno, se toma el de mayor severidad. Aunque resuelvas varios en la misma tanda, cada item mantiene su propio plan, su propio diff y su propia prueba — no se mezclan entre sí, para no perder de vista cuál corrección resuelve cuál hallazgo.

## Paso 1 — Generar el plan (no tocar código)

Autenticación · auth/ **API de requerimientos · api/** Interfaz web · web/

```
AMBITO
Workspace: ai-labs, abierto en su raiz. Todas las rutas son relativas
a esa raiz. Lee el codigo solo de api/; escribe codigo, configuracion y
pruebas solo en api/. No leas ni modifiques codigo de otros componentes.
Lee README.md, RETO.md y docs/GOOD_PRACTICES.md. Lee
HALLAZGOS.md, PLAN.md y los demas documentos de docs/ solo si existen.
Escribe evidencias solo en HALLAZGOS.md, PLAN.md, prompts/,
docs/ENDPOINTS.md, docs/TESTING.md y docs/GOOD_PRACTICES.md de la raiz.
Cuando este paso pida un artefacto ausente, crealo. Si ya existe, conserva
las filas y secciones de otros componentes. No las sobrescribas.
```

```
Cada hallazgo lleva componente api e ID H-API-NN, por ejemplo H-API-01.
Cita codigo como api/ruta:linea. No renumeres IDs existentes.
Ejecuta Git desde ai-labs/; ejecuta pruebas con cwd api/.
Coordina dependencias externas sin ampliar AMBITO. No inventes aprobaciones
humanas ni resultados de pruebas. Si falta evidencia, registra el pendiente.

CONTEXTO
Trabaja solo las filas y secciones de api en los documentos generales.
```

Antes de pegar el siguiente bloque, una persona revisa PLAN.md y registra aprobador, fecha, H-API-NN seleccionados, contratos y pruebas en ese archivo. La IA no puede darse esa aprobacion. Si afecta a otro componente, registra la dependencia, coordina su responsable y solicita un ambito explicito aprobado. Conserva los limites del AMBITO actual hasta entonces.

## Paso 2 — Implementar el plan y cerrar los items

Autenticación · auth/ API de requerimientos · api/ Interfaz web · web/

```
AMBITO
Workspace: ai-labs, abierto en su raiz. Todas las rutas son relativas
a esa raiz. Lee el codigo solo de api/; escribe codigo, configuracion y
pruebas solo en api/. No leas ni modifiques codigo de otros componentes.
Lee README.md, RETO.md, HALLAZGOS.md, PLAN.md y
docs/GOOD_PRACTICES.md. Consulta los demas documentos de docs/ si existen.
Escribe evidencias solo en HALLAZGOS.md, PLAN.md, prompts/,
docs/ENDPOINTS.md, docs/TESTING.md y docs/GOOD_PRACTICES.md de la raiz,
```

```
conservando las filas y secciones de otros componentes. No los sobrescribas.
Cada hallazgo lleva componente api e ID H-API-NN, por ejemplo H-API-01.
Cita codigo como api/ruta:linea. No renumeres IDs existentes.
Ejecuta Git desde ai-labs/; ejecuta pruebas con cwd api/.
Coordina dependencias externas sin ampliar AMBITO. No inventes aprobaciones
humanas ni resultados de pruebas. Si falta evidencia, registra el pendiente.

CONTEXTO
Trabaja solo las filas y secciones de api en los documentos generales.
Conserva contenido previo de otros componentes y el historial de este componente.
```

Si `HALLAZGOS.md` todavía tiene items "Pendiente" que quieras resolver, vuelve a ejecutar este mismo prompt indicando cuáles.

✓ Completado

CIERRE · Paso 9 de 9

# Cerrar y entregar al Día 2

Revisa evidencia y pruebas reales, integra day1/prompting a main con revisión humana y crea checkpoint/day1 local.

## Cierre del día: handoff al Día 2

Conserva en la raíz `HALLAZGOS.md`, `PLAN.md`, `prompts/*.md`, `docs/ENDPOINTS.md`, `docs/TESTING.md` y `docs/GOOD_PRACTICES.md`, junto con código y tests dentro de cada componente trabajado. Los prompts pueden nombrarse

`prompts/auth-discovery.md`, `prompts/api-refactor.md` o `prompts/web-testing.md`. Mantén IDs globales estables, componente y evidencia con rutas desde la raíz.

Deja pendientes reales con motivo y siguiente acción. Si resolviste todos, no inventes uno para aprobar: el instructor aporta HU de referencia al Día 2. La evaluación revisa calidad por separado del avance declarado en el portal.

Desde la raíz, en `day1/prompting`, revisa todo el cambio y los archivos nuevos con la persona revisora:

```
>_ Línea de comandos · bash

git status --short
git diff main --stat
git diff main
```

Un diff no muestra archivos sin seguimiento; inspecciónalos también. Excluye `.env`, `.venv`, bases SQLite, uploads, `node_modules`, `target` y reportes de cobertura del commit. Incluye solo rutas revisadas de código y tests del componente elegido y los documentos generales modificados. Por ejemplo, si trabajaste autenticación:

```
>_ Línea de comandos · bash

git add -- auth HALLAZGOS.md PLAN.md prompts docs
git diff --cached --check
git diff --cached
git commit -m 'day1: prompts, correcciones, pruebas y documentación'
```

Para API o frontend, cambia `auth` por `api` o `web`; si trabajaste varios, agrega sus rutas deliberadamente. Verifica el staged antes del commit. Incluye cualquier otro archivo general modificado solo por su nombre, después de revisarlo.

La revisión humana del plan y del diff final, con persona, fecha, IDs y veredicto, debe quedar registrada realmente en `PLAN.md` o `HALLAZGOS.md` antes de integrar. Si quedan bloqueantes o pruebas exigidas sin ejecutar, conserva `day1/prompting` abierta y registra el motivo. La IA puede proponer un veredicto, pero no reemplazar a quien aprueba.

Solo después de esa revisión y de las pruebas reales, la persona responsable integra localmente a `main`:

```
>_ Línea de comandos · bash
git switch main
git merge --no-ff day1/prompting -m 'Integra Día 1 con revisión humana'
git status --short
```

Si hay conflictos, resuélvelos con revisión humana y repite las pruebas afectadas antes de seguir. No crees el checkpoint mientras quede un merge en curso, cambios sin guardar o verificaciones bloqueantes. Confirma la integración y consulta la evidencia guardada:

```
>_ Línea de comandos · bash
git merge-base --is-ancestor day1/prompting main
git log -1 --oneline main
```

`git merge-base` debe terminar con código 0. El estado de trabajo debe estar limpio y la evidencia debe describir las pruebas realmente ejecutadas. Solo entonces crea la rama de checkpoint y cierra la rama de trabajo integrada:

```
>_ Línea de comandos · bash
git branch checkpoint/day1 main
```

```
git branch -d day1/prompting
```

```
git branch --list main checkpoint/day1
```

```
git rev-parse main checkpoint/day1
```

Ambas referencias deben apuntar al mismo commit en este cierre. Si `checkpoint/day1` ya existe, inspecciona su commit con el instructor; no la fuerces ni la muevas automáticamente. Un checkpoint existente tampoco sustituye la evidencia. El cierre usa solo ramas locales, sin tags, push ni PR de GitHub.

Entrega la ruta local, SHA de `main` y `checkpoint/day1`, componentes trabajados, documentos y pendientes. El verificador de artefactos conserva compatibilidad con materiales legacy; comprueba sus requisitos por separado. Su resultado no acredita ejecución ni revisión humana y no sustituye las evidencias de este cierre. Los días 2 a 4 deben adaptar sus rutas e IDs a esta raíz compartida antes de continuar. Una actividad futura que publique en GitHub necesita preparación y autorización propias; este día no crea ese remoto.

# Evaluación D1: prompting y Secure AI

Puerta de entrada: preparación local de `ai-labs`. Umbral de salida: **promedio ponderado ≥ 8.0**, revisión humana real y rama local `checkpoint/day1` creada después de integrar a `main` y comprobar las evidencias.

| # | CRITERIO | PESO | APRUEBA CON 8 O MÁS SI... | EVIDENCIA |
|---|---|---|---|---|
| 1 | Entregables | 30% | `HALLAZGOS.md`, `PLAN.md`, `prompts/`, `docs/ENDPOINTS.md`, `docs/TESTING.md` y `docs/GOOD_PRACTICES.md` están en la raíz, con código y tests en los componentes trabajados; `day1/prompting` se integra y cierra con revisión humana | SHA local de `main` y `checkpoint/day1`, historial de integración, documentos y pruebas ejecutadas |
| 2 | Contenido | 30% | Hay pendientes reales con siguiente acción, o resolución completa justificada con HU de referencia para D2; contrato y pruebas son coherentes con código; sin secretos ni datos reales | `HALLAZGOS.md` general, diff revisado y `docs/TESTING.md` |

| # | CRITERIO | PESO | APRUEBA CON 8 O MÁS SI... | EVIDENCIA |
|---|---|---|---|---|
| 3 | Trazabilidad | 20% | Cada hallazgo tiene componente e ID global `H-AUTH-NN`, `H-API-NN` o `H-WEB-NN`, con evidencia `componente/ruta:línea`; `PLAN.md` referencia esos IDs | Referencias cruzadas en los documentos generales |
| 4 | Criterio humano | 20% | El review confirma o reabre con motivo; una persona aprueba el plan y el diff final con nombre, fecha, alcance y veredicto reales | Registro humano en `PLAN.md` o `HALLAZGOS.md`; la salida de la IA no lo sustituye |

Escala por criterio: 0 ausente; 1 a 3 parcial; 4 a 5 básico; 6 a 7 sólido; 8 a 10 ejemplar y verificable.

La rama inicial es `day1/prompting` desde `main`, una vez para todo el monorepo. La revisión ejecuta Git desde la raíz e incluye `git diff main`, estado y archivos nuevos. Los comandos de pruebas usan cwd `auth/`, `api/` o `web/` y registran resultados reales. No se crea `checkpoint/day1` si faltan evidencias o hay bloqueantes; una rama existente no demuestra calidad ni ejecución.

El cierre es local y usa ramas. El verificador de artefactos no sustituye las pruebas ni la revisión humana. Los tags del recorrido legacy no son un gate de este flujo. No se exige GitHub, push ni PR remoto. Puedes completar uno, dos o los tres componentes; documenta también las integraciones no probadas.

Para evaluar prompting revisa el pedido ambiguo, su adaptación de seis partes, la justificación de criterios, una afirmación contrastada con código y pruebas y el seguimiento ante evidencia faltante. Guarda esos prompts en `prompts/` general, con el componente en su nombre.

Leer, copiar o descargar el material